APS2 izpitni povzetek v2 - ena A4 stran

Asimptotika

- O(g): obstajata c,n0, da f(n)<=c*g(n) za n>=n0. Omega je spodnja, Theta oboje.
- Polinomi: vzame se najvisja potenca. Logaritmi so pocasnejsi od n^eps, eksponenti hitrejsi od polinomov.
- Vsota zank: zaporedno sestejemo, gnezdeno pomnozimo, pri deljenju problema nastane log n nivojev.
- Master: aT(n/b)+f(n). Primerjaj f(n) z n^log_b(a).

ADT in tabele

- ADT opise operacije in obnasanje, implementacija pove strukturo in cene.
- Tabela: dostop O(1), iskanje neurejeno O(n), urejeno O(log n), vrivanje v sredino O(n).
- Dinamicna tabela z razsirjanjem x2 ima amortizirano vrivanje O(1). Potencial meri placano prihodnje kopiranje.
- Pri nalogi vedno povej, kateri indeks se premakne in koliko elementov se kopira.

Seznami in skip list

- Povezani seznam: vrivanje po znanem kazalcu O(1), iskanje O(n), slab cache locality.
- Dvojno povezan seznam omogoca brisanje znanega vozlisca brez iskanja predhodnika.
- Skip list: nivo vozlisca je nakljucen. Pricakovano iskanje, vstavljanje, brisanje O(log n).
- Postopek iskanja: desno dokler ne presezes kljuca, nato dol.

Slovarji in hash

- Dobra zgosc. funkcija enakomerno razprsi kljuce. Load factor alpha=n/m.
- Verizenje: pricakovano O(1+alpha). Odprto naslavljanje: sondiranje, brisanje potrebuje tombstone.
- Linearno sondiranje povzroca primary clustering; kvadratno/dvojno zmanjsata kopicenje.
- Bloom: vstavljanje nastavi k bitov, iskanje preveri vseh k bitov. Ni laznih negativnih, so lazni pozitivni.

Kopice

- Min/max heap je skoraj polno binarno drevo v tabeli. Stars (i-1)//2, otroka 2i+1,2i+2.
- insert: dodaj na konec in sift-up O(log n). delete-min/max: zamenjaj s koncem in sift-down O(log n).
- build-heap od spodaj je O(n), ker je veliko vozlisc blizu listov.
- Heapsort: build O(n), n izbrisov O(n log n), in-place, obicajno nestabilen.

Drevesa

- BST inorder izpise urejene kljuce. Cena je O(h); uravnotezeno h=O(log n), degenerirano h=O(n).
- AVL: visinska razlika najvec 1. Rotacije LL, RR, LR, RL popravijo lokalno neravnovesje.
- B-drevo: vec kljucev v vozliscu, vsi listi enako globoki, primerno za diske/bloke.
- Pri rotaciji napisi nov koren poddrevesa in prestavi srednje poddrevo na pravo stran.

DSU

- make-set, find, union. Union by rank/size + path compression da skoraj konstantno amortizirano ceno.
- Kruskal: sortiraj povezave po tezi, dodaj povezavo, ce find(u)!=find(v), nato union.
- Cikla ne dodamo, ker sta krajisci ze v isti komponenti.

Dinamicno programiranje

- Stanja morajo zajeti tocno dovolj zgodovine. Prehod gradi optimalno resitev iz manjsih podproblemov.
- Rod cutting: dp[i]=max(cena[j]+dp[i-j]). Hrani izbiro j za rekonstrukcijo.
- Nahrbtnik 0/1: dp[i][w]=max(dp[i-1][w], vred[i]+dp[i-1][w-teza[i]]).
- LCS: enaka znaka -> 1+dp[i-1][j-1], sicer max(dp[i-1][j],dp[i][j-1]).
- Pri izpitu zapisi stanje, zacetne pogoje, prehod, vrstni red racunanja in rekonstrukcijo.

Grafi

- BFS: nivoji, najkrajse poti v neutezenem grafu, O(V+E). Vrsta.
- DFS: casi vstop/izstop, topolosko urejanje, komponente, cikli. Sklad/rekurzija.
- Dijkstra: nenegativne utezi, prioritetna vrsta, O((V+E)logV). Bellman-Ford dopusca negativne utezi.
- Topoloski sort obstaja samo za DAG. Kahn: vozlisca z vhodno stopnjo 0.
- MST: Kruskal sortira povezave, Prim raste iz vozlisca. Oba uporabljata lastnost reza.

P/NP in redukcije

- P: resljivo polinomsko. NP: resitev preverimo polinomsko. NP-tezko: nanj reduciramo NP probleme.
- NP-polno: v NP in NP-tezko. Redukcija A <=p B pomeni: ce znam B, znam tudi A.
- Dokaz NP-polnosti: clan v NP, znan NP-poln problem, preslikava, dokaz ekvivalence.

Pasti

- Ne mesaj najkrajse poti in MST: MST ne daje najkrajsih poti med pari.
- Dijkstra odpove pri negativnih povezavah. DFS ne daje najkrajsih poti v splosnem grafu.
- Pri hashanju vedno navedi nacin resevanja trkov. Pri DP ne pozabi zacetnih pogojev.
- Rezultat brez postopka je nevaren: pri vsaki nalogi napisi invariant ali razlog pravilnosti.